~/Library/CloudStorage/OneDrive-NationalUniversityofSingapore/NUS BIT_OneDrive/TCX1002_IntroProgramming/intropgm_tut2.py

```
#%% Qn1 - Dean's list
import math

Sem2_results = [["Aaron", "A", "B+", "B"], ["Ben", "B", "B+", "C"], ["Carol", "A", "A", "A"], ["David", "B", "B", "B"], ["Eve", "A", "A", "A-"], ["Fen", "A", "B", "C"], ["Gordon"
, "B", "C", "D"], ["Hannah", "C", "C","C"], ["Ian", "A", "A"], ["John", "A","A","A-"]]

grades_dict = {"A":5,"A-":4.5,"B+":4,"B":4,"B-":3,"C+":2.5,"C":2,"D":1,"F":0}

def get_deans_list(Sem2_results):
  stud_res_list = student_results(Sem2_results)
  stud_res_list.sort(key= lambda x:x[1], reverse=True)
  # print(stud_res_list)
  dean_list_len = math.ceil(len(stud_res_list)*0.2)
  dean_list = []
  threshold_gpa = stud_res_list[dean_list_len-1][1]
  # print(threshold_gpa)
  dean_list=[name for name,gpa in stud_res_list if gpa>=threshold_gpa]

  return dean_list

def student_results(Sem2_results):
  eligible_list = []
  for entry in Sem2_results:
    if len(entry)<4:
      continue
    grades = entry[1:]
    sum = 0
    for grade in grades:
      sum += grades_dict[grade]
    gpa = f"{(sum/3.0):.2f}"
    eligible_list.append([entry[0],gpa])
  return eligible_list


get_deans_list(Sem2_results)==['Carol', 'Eve', 'John']


#%%
Sem2_results = [["Aaron", "A", "B+", "B"], ["Ben", "B", "B+", "C"], ["Carol", "A", "A", "A"],
                ["David", "B", "B", "B"], ["Eve", "A", "A", "A-"], ["Fen", "A", "B", "C"],
                ["Gordon", "B", "C", "D"], ["Hannah", "C", "C","C"], ["Ian", "A", "A"], ["John", "A","A","A-"]]

grades_dict = {"A":5,"A-":4.5,"B+":4,"B":4,"B-":3,"C+":2.5,"C":2,"D":1,"F":0}

def get_deans_list(Sem2_results):
  stud_res_list = student_results(Sem2_results)
  stud_res_list.sort(key= lambda x:x[1], reverse=True)
  dean_list_len = math.ceil(len(stud_res_list)*0.2)
  dean_list = []
  threshold_gpa = stud_res_list[dean_list_len-1][1]
  dean_list=[name for name,gpa in stud_res_list if gpa>=threshold_gpa]

  return dean_list

def student_results(Sem2_results):
  eligible_list = []
  for entry in Sem2_results:
    if len(entry)<4:
      continue
    grades = entry[1:]
    sum = 0
    for grade in grades:
      sum += grades_dict[grade]
    gpa = f"{(sum/3.0):.2f}"
    eligible_list.append([entry[0],gpa])
  return eligible_list



#%% Qn2 - Olympic Medal Table
```

```
rows = [
  ("CountryA", 1, 3, 0),
  ("CountryC", 2, 4, 1),
  ("CountryB", 2, 4, 1),
  ("CountryD", 4, 3, 5),
  ("CountryE", 1, 7, 2)
]

def rank_medals(rows):
  sorted_rows = sorted(rows, key=lambda row: (-row[1], -row[2],-row[3], row[0]))

  rank = 0
  prev_medals = None

  ranked_res = []

  for country,g,s,b in sorted_rows:
    medals = (g,s,b)

    if medals!=prev_medals:
      rank+=1
      prev_medals = medals
    ranked_res.append((rank,country,g,s,b))

  return ranked_res
rank_medals(rows)

#%% Qn3 - v1 "doesnt work"

def bar_chart(values):
  height = max(values)
  col_width = max([len(str(v)) for v in values])
  spaces = ' ' * col_width
  res = []
  v_string = (" ".ljust(col_width)).join((str(num) for num in values))

  for row in range(height):
    temp = []
    for v in values:
      if v>row:
        temp.append(chr(9608).ljust(col_width))
      else:
        temp.append(' '*col_width)
    row_res = (spaces.join(temp))
    res.append(row_res)

  res = res[::-1]
  res.append(v_string)
  return res

# print(bar_chart(values))

# print(bar_chart([3,0,12,7,3])[9]=='▉   ▉ ▉ ▉ ')
print(len('▉   ▉ ▉ ▉ '))
bar_chart([3,0,12,7,3])
# bar_chart([4])

#%% Qn3 - v2 (works)
def bar_chart(values):
  height = max(values)
  col_width = len(str(height))

  res = []
  for row in range(height):
    temp = []
    for v in values:
      if v > row:
        # Use a solid block of the correct width
        temp.append(chr(9608).ljust(col_width))
      else:
        # Use spaces of the correct width
```

```
        temp.append(' ' * col_width)
    # Join columns with a single space gap
    res.append(' '.join(temp))

  res = res[::-1]

  # Format labels: each number left-aligned to col_width, joined by a space
  v_string = ' '.join(str(v).ljust(col_width) for v in values)
  res.append(v_string)

  return res

bar_chart([3,0,12,7,3])[9]
'▇   ▇ ▇ ▇'

#%% Qn4
'''Write a function to merge two already-sorted iterables into a single iterable, using a custom comparator comp(x, y) that returns:
< 0 if x < y
0 if x == y
> 0 if x > y
Examples:
tuple(merge(range(1,5,2), range(2,5))) should return (1, 2, 3, 3, 4)
list(merge([(3,1),('abc',6),('y',7)], [('a',1),(2.4, 3),('km',9)], lambda x, y: x[1]-y[1])) should return [(3, 1), ('a', 1), (2.4, 3), ('abc', 6), ('y', 7), ('km', 9)]
"".join(merge("ace", 'bd', lambda x, y: ord(x)-ord(y))) should return 'abcde'
Note, you are not allowed to use Python's built-in sorting functions.
'''
def merge(it1, it2, comp = lambda x, y:x-y):
  it1 = iter(it1)
  it2 = iter(it2)
  pass


#%%
discount_list = {
  'laptop': [(1, 0), (5, 10), (10, 20)],
  'tablet': [(1, 0), (3, 5)],
  'smartphone': [(1, 0), (3, 10)]
}

def calculate_price(items, standard_price):
  # grand_total = [(standard_price[x[0]] * x[1]) for x in items]
  res = {}
  if 'items' and 'total_discounted' not in res.keys():
    res['total_discounted'] = 0
    res['items'] = []

  for item in items:
    total = standard_price[item[0]] * item[1]
    item_name = item[0]
    print('item name: ' + str(item_name))
    print('item total: ' + str(total))
    qty_purchased = item[1]
    print('qty purchased: ' + str(qty_purchased))
    discount = max([x[1] for x in discount_list[item[0]] if qty_purchased >= x[0]])
    print('discount: ' + str(discount))
    grand_total = int(total * (100-discount)/100)
    print(int(grand_total))

    res['items'].append((item_name, qty_purchased, grand_total))
    res['total_discounted'] += int(grand_total)

  return res

calculate_price([('laptop', 12), ('tablet', 4), ('smartphone', 2)],{'laptop': 1000, 'tablet': 500, 'smartphone': 300} )

#expected: {  'total_discounted': 12200,  'items': [('laptop', 12, 9600), ('tablet', 4, 2000), ('smartphone', 2, 600)] }


#%% Qn6 - Sqrt(n)

'''
1. start with guess x
```

```
2. improve using x_next = 0.5*(x + (n/x))
- while cond1 && cond2
  - improve
- else: return values
'''

def sqrt_iter(n: float, tol: float = 1e-10, max_iter: int = 10000) -> float:
  count = 0
  diff = 9999999999999999
  x = n

  try:
    while (diff > tol) and (count <= max_iter) and n>0:
      x_next = 0.5*(x + (n/x))
      diff = abs(x_next - x)
      x = x_next
      count += 1

    # return x, diff, count
    return x
  except:
    return "Error"

sqrt_iter(0)
# abs(sqrt_iter(2.0)-1.414213)<1e-5

#%% Qn7 - Iterative Search in a Binary Search Tree

'''In this exercise, we represent a Binary Search Tree (BST) using a Python list. Each element in the list is a tuple: (value, left_index, right_index) where:
value — the integer stored at the node
left_index — the index of the left child in the list (or -1 if none)
right_index — the index of the right child in the list (or -1 if none)
You are tasked to write a function which searches for target value in the tree. Return the index of the matching node if found, otherwise return -1. You must use iteration, not recursion.
'''

def bst_search(tree, target, root_index=0):
  try:
    return [i for i in range(root_index,len(nodes)) if nodes[i][0]==target][0]
  except:
    return -1

# do NOT modify these test cases:
nodes = [(8, 1, 2), (3, -1, 4), (10, 3, -1), (9, -1, -1), (5, -1, -1)]

print(bst_search(nodes, 10))  # should be 2
print(bst_search(nodes, 5))   # should be 4
print(bst_search(nodes, 7))   # should be -1

# other edge cases:
# - more than 1 nodes have the same target value

#%% Qn8 - Rank Days by Moving-Average Increase

'''You are given a list of daily stock price for company X. You should first compute a trailing simple moving average (SMA) with a configurable window size, then return the indices of days sorted by the day-to-day increase in that SMA (largest increase first).
A trailing simple moving average (SMA) is the average of the most recent window values in a time series, where each average is aligned to the latest data point in that window.
For example, given:
prices = [50,  52,  51,  53,  55,  56,  59,  59], window size is 3. Then:
SMA = [None, None, 51, 52, 53, 54.67, 56.67, 58.0]
inc =[None, None, None, 1,  1,  1.67, 2, 1.33]
should finally return indices: [6, 5, 7, 3, 4]. We only check the final returnedindices, the intermediate calculations are for your information only.
'''

# prices = [50,  52,  51,  53,  55,  56,  59,  59]

def avg_window(prices, window, i):
  sum = 0
  if i < window-1:
    return None
  for j in range(window):
    sum += prices[i]
```

```
    i -= 1
  return float(sum/window)
  # return float("%.5f" % (sum/window))

def calc_inc(SMA):
  res = []
  for i in range(len(SMA)):
    try:
      inc = SMA[i]-SMA[i-1]
      res.append(float("%.5f" % inc))
    except:
      res.append(None)
  return res


def rank_sma_increases(prices, window):
  SMA = []
  for i in range(len(prices)):
    SMA.append(avg_window(prices, window, i))

  # print(f"SMA: {SMA}")
  # print(f"Inc: {calc_inc(SMA)}")
  inc_list = list(enumerate(calc_inc(SMA)))
  clean_inc_list = [x[0] for x in (sorted([e for e in inc_list if e[1]!=None], key=lambda item:item[1] ,reverse=True))]
  print(sorted([e for e in inc_list if e[1]!=None], key=lambda item:item[1] ,reverse=True))
  return clean_inc_list

rank_sma_increases([50, 52, 51, 53, 55, 56, 59, 60], window=3)
# rank_sma_increases([50, 52, 51, 53, 55, 56, 59, 60], window=4)

#%% Qn8 - sliding window approach

'''pseudocode:
1. calc SMA: generate average, padding with None when window is incomplete
2. calc Differences: SMA[i]-SMA[i-1]
3. Rank indices: sort out differences according to rank, filtering out None values
'''

def rank_sma_increase(prices, window):
  #1. calc SMA using list slicing
  sma = []
  for i in range(len(prices)):
    if i < window-1:
      sma.append(None)
    else:
      window_slice = prices[ i-window+1 : i+1 ]

  pass
```